

常州工学院

CHANGZHOU INSTITUTE OF TECHNOLOGY

课程大作业说明书

课程名： 大数据与云计算

题 目： 电商行为大数据分析系统

二级学院： 计算机信息工程学院

专 业： 软件工程

班 级： 23 软件一

学号姓名： 23030327 许子祺

23030315 莫文涛

23030316 钱季清

23250123 王奕泽

指导教师： 李亦飞

2026 年 6 月

摘 要

本文围绕电商用户行为数据分析课题，构建了一套基于 Docker ARM64、Hadoop HDFS/YARN、Spark SQL、Flask API 与 React 前端的本机大数据分析实验系统。系统以 Kaggle 电商行为数据为主要输入，完成原始 CSV 数据采样、HDFS 上传、Spark 分布式读取、字段标准化、异常值过滤、重复事件去重、会话事实表生成、指标缓存发布和前端可视化展示。业务层面实现了基础看板、转化漏斗、用户行为路径、特征集市、需求预测、商品关联图谱、Cohort 留存、商品组合分析、购物车挽回、收入归因、运营优化、个性化推荐、异常检测、生命周期分层、实验分流和 Ops 证据面板等模块。各模块统一复用清洗后的事件表、会话事实表和日粒度特征表，保证不同页面中的行数、收入、转化率、留存率、推荐置信度和异常分级口径一致。

在工程实现上，本文重点解决了本机 ARM Mac 环境下 Hadoop/Spark 集群复现、端口冲突、跨架构镜像不可用、Spark History 日志过大、driver 端全量 collect 造成内存风险等问题。系统采用 YARN client mode 作为第一阶段提交方式，保留 Flask 读取本地缓存的便利性；同时使用 AQE、合理 shuffle 分区、MEMORY_AND_DISK 缓存、Parquet 清洗层、preview JSON 上界和模块级质量门禁降低运行风险。实验部分对比 local Spark、YARN-only、YARN+AQE、YARN+ 算法限流和 YARN+Parquet 多组配置，并补充 1%/5% 用户级样本与典型模块 benchmark。结果表明，仅把任务提交到 YARN 并不能自动提升性能或避免 OOM；真正有效的优化来自 Spark SQL 执行计划、列式数据布局、算法输出边界和可追溯的运行证据。最终系统能够在课程设计环境中完成从分布式存储、Spark 计算、后端接口到前端可视化和报告分析的完整闭环。

关键词：Hadoop YARN；Spark SQL；Docker ARM64；电商行为分析；内存优化；Benchmark

Abstract

This report presents a local big-data analytics system for ecommerce behavior analysis, built with Docker ARM64, Hadoop HDFS/YARN, Spark SQL, Flask APIs, and a React frontend. The system uses the Kaggle ecommerce behavior dataset as the main input and implements CSV sampling, HDFS ingestion, Spark-based cleansing, event normalization, anomaly filtering, duplicate removal, session fact generation, metric cache publishing, and frontend visualization. At the business layer, the project covers dashboard metrics, conversion funnels, customer journeys, a behavior feature mart, demand forecasting, product affinity graphs, cohort retention, portfolio analysis, cart recovery, revenue attribution, merchandising optimization, personalized recommendation, anomaly detection, lifecycle segmentation, experimentation, and an Ops evidence panel. These modules share the same cleaned event table, session facts, and daily feature tables, so that row counts, revenue, conversion rates, retention rates, recommendation confidence, and anomaly severity remain consistent across pages.

From an engineering perspective, the project focuses on reproducibility and stability in a local ARM Mac environment. It addresses Hadoop/Spark cluster setup, port conflicts, unavailable amd64 Hadoop images, oversized Spark History event logs, and driver-side memory risks caused by unbounded collection. The system uses YARN client mode in the first stage so that Flask can still read local cache outputs, while AQE, controlled shuffle partitions, MEMORY_AND_DISK caching, a Parquet cleansing layer, bounded preview JSON, and module-level quality gates are used to reduce operational risk. Benchmarks compare local Spark, YARN-only, YARN+AQE, YARN+algorithm guards, and YARN+Parquet configurations on 1% and 5% user-level samples, together with module-level benchmark evidence. The results show that submitting a job to YARN alone does not automatically improve performance or eliminate OOM risk. Effective optimization requires Spark SQL tuning, columnar data layout, bounded algorithm outputs, and traceable runtime evidence. The final system forms a complete course-design workflow from distributed storage and Spark computation to backend APIs, frontend visualization, and report analysis.

Keywords: Hadoop YARN; Spark SQL; Docker ARM64; Ecommerce Analytics; Memory Optimization; Benchmark

目 录

第 1 章 绪论（课题背景、现状）	1
1.1 课题背景	1
1.2 国内外现状	1
1.3 课题问题定义	2
1.4 本文结构	2
第 2 章 相关技术（系统使用技术介绍）	3
2.1 Docker ARM64 实验环境	3
2.1.1 Docker、HDFS、YARN 与 Spark 的协同关系	4
2.2 HDFS 分布式存储	5
2.3 YARN 资源调度	6
2.4 Spark SQL 与 DataFrame API	7
2.5 Spark AQE 与执行计划优化	8
2.6 Parquet 与数据布局	8
2.7 内存优化与数据倾斜控制	9
2.8 Flask API 服务层	9
2.9 React、ECharts 与前端可视化	10
2.10 Ops 证据面板与可追溯性	11
2.11 SQLite 作业记录与缓存文件	11
第 3 章 需求分析（设计目标）	12
3.1 总体目标	12
3.2 功能需求	12
3.2.1 数据处理功能	12
3.2.2 集群运行功能	12
3.2.3 运维展示功能	12
3.3 非功能需求	13
3.3.1 架构兼容性	13
3.3.2 内存稳定性	13
3.3.3 证据可追溯性	13
3.4 实验设计	13
3.5 验收指标	13

3.6 数据清理需求	14
第 4 章 系统设计与实现（实现步骤截图、环境安装、代码实现）	15
4.1 总体架构	15
4.2 环境安装与启动步骤	16
4.2.1 ARM64 Hadoop 镜像构建	16
4.2.2 Docker Compose 启动	16
4.2.3 HDFS 目录准备	17
4.3 Spark 提交链路实现	18
4.3.1 YARN client 配置	18
4.3.2 提交脚本	18
4.3.3 YARN cluster mode 入口	19
4.4 核心算法与内存优化实现	19
4.4.1 禁止 full collect	19
4.4.2 WindowExec 警告优化	19
4.4.3 关联分析组合膨胀控制	20
4.4.4 Parquet 清洗层	20
4.5 核心业务模块数据计算方法	20
4.5.1 基础清洗与元数据口径	21
4.5.2 模块计算总览	21
4.5.3 基础看板与排行模块	23
4.5.4 转化分析模块	24
4.5.5 行为路径模块	24
4.5.6 特征集市模块	25
4.5.7 预测模块	26
4.5.8 关联图谱模块	26
4.5.9 Cohort 留存模块	27
4.5.10 商品组合与购物车挽回模块	27
4.5.11 收入归因模块	28
4.5.12 运营优化与推荐模块	28
4.5.13 异常检测与生命周期模块	29
4.5.14 实验分流与 Ops 证据模块	30
4.6 前端 Ops 页实现	30
4.7 旧数据清理实现	31

4.8 实验结果	32
4.8.1 Benchmark 对照表	32
4.8.2 Spark History 指标	33
4.8.3 典型模块 Benchmark	33
4.8.4 结果分析	34
4.8.5 实验结论	35
第 5 章 致谢	36
参考文献	37

第1章 绪论（课题背景、现状）

1.1 课题背景

电子商务平台每天会产生大量用户行为日志，包括浏览、加购、移除购物车、购买、商品、类目、品牌、价格、用户和会话等字段。单机脚本可以完成小样本统计，但当数据量从几十 KB 的演示文件增长到数 GB 的真实日志时，原有处理方式会暴露出三个问题：第一，CSV 扫描成本高，重复读取会拖慢整个分析流程；第二，复杂分析模块容易把明细结果收集到 driver 端，造成 Python list 或 JSON payload 过大；第三，关联分析、推荐候选、异常检测等模块存在组合膨胀风险，数据规模增加后可能出现长尾 task、shuffle 放大和内存溢出。

本课题的目标不是简单把程序放进 Docker，也不是仅把 Spark master 改成 YARN，而是围绕“真实大数据实验是否可复现、可解释、可对比”设计一套本机稳妥型实验系统。系统使用 Kaggle 电商行为数据作为原始输入，保留 Oct 和 Nov 两个月 CSV 原始数据，通过用户级采样构造 1% 与 5% 实验集，再在 Docker ARM64 Hadoop YARN 集群上运行 PySpark pipeline。最终产出包括 HDFS 数据快照、Spark 运行记录、质量报告、前端 Ops 页和 benchmark 对比结果。

1.2 国内外现状

大数据批处理的经典起点是 MapReduce。MapReduce 将海量数据处理抽象为 Map 和 Reduce 两个阶段，通过分布式文件系统、任务重试和数据本地性降低大规模批处理的工程复杂度^[1]。Hadoop 生态在此基础上形成了 HDFS、MapReduce 和 YARN 等组件。YARN 将资源管理从具体计算框架中抽离出来，由 ResourceManager、NodeManager 和 ApplicationMaster 协同完成资源调度，使同一集群能够承载不同计算引擎^[2]。

Spark 在 RDD 模型中引入内存计算、lineage 容错和数据复用，解决了 MapReduce 在迭代计算、交互式分析和多阶段作业中的效率问题^[3-4]。Spark SQL 又通过 Catalyst 优化器、DataFrame API、列式执行和谓词下推，使结构化数据分析更加接近数据库优化器的工作方式^[5]。在现代 Spark 应用中，是否启用 AQE、是否使用 Parquet、是否合理控制 shuffle 分区和 driver collect，往往比“是否使用集群”本身更影响稳定性。

同时，学术界和工业界已经认识到大数据系统性能不能只依赖静态经验参数。

Starfish 提出 `profile-driven tuning`, 强调先收集运行画像, 再根据证据调优^[6]; `Skew-Tune` 关注数据倾斜和长尾 `task` 对整体作业时间的影响^[7]; `Themis` 则提醒系统设计不能盲目追求全内存, 应当接受合理 `spill` 和磁盘中间结果^[8]。这些思想与本课题的实验结论一致: 仅增加 `YARN` 不能自动避免 `OOM`, 必须把资源调度、执行计划、数据布局和算法输出边界一起纳入设计。

1.3 课题问题定义

本项目早期使用 `local Spark` 和小样本 `JSON` 缓存, 适合演示页面功能, 但不足以支撑“大数据与云计算”课程中的集群实验和论文结论。前端出现红色长条警告、滚动异常等问题时, 根因并不只在页面布局, 也与后端一次性输出过多明细、质量状态不清晰和运行证据分散有关。因此, 本课题将问题拆成四类:

1. **数据规模问题**: 从 87.4KB `smoke` 文件扩展到真实 `Oct/Nov` 原始 `CSV`, 并构造 1% 和 5% 用户级样本。
2. **运行环境问题**: 从 `local[*]` 升级为 `Docker Hadoop YARN` 实验集群, 同时兼容 `ARM64 Mac`, 避免下载 `amd64 Hadoop` 镜像。
3. **算法内存问题**: 限制 `driver` 端 `full collect`, 将明细输出收敛为 `Parquet` 与 `preview JSON`, 并对关联、推荐、异常等模块加入上界。
4. **证据展示问题**: 把 `benchmark`、`HDFS` 输入、`YARN app id`、质量状态和清理策略汇总到前端 `Ops` 页与 `LaTeX` 报告中, 减少答辩时翻终端的成本。

1.4 本文结构

本文第 2 章介绍 `Hadoop`、`YARN`、`Spark`、`Parquet`、`Docker` 和前端运维展示等相关技术; 第 3 章给出系统需求、设计目标和验收指标; 第 4 章详细说明 `Docker YARN` 集群、`Spark` 提交流程、核心代码实现、数据清理、前端展示和 `benchmark` 实验结果; 第 5 章为致谢; 最后列出参考文献。

第2章 相关技术（系统使用技术介绍）

2.1 Docker ARM64 实验环境

本项目的实验环境不是直接在宿主机上安装 Hadoop 与 Spark，而是使用 Docker Compose 组织一套可重复启动、可清理、可展示的本机实验集群。这样设计的原因有三点。第一，Hadoop、Spark、Java、Python、Flask 和前端 Node 环境依赖较多，如果全部安装在宿主机上，版本冲突和路径污染会影响复现。第二，课程答辩需要展示 HDFS、YARN、Spark History 等服务的运行状态，容器化可以把这些服务固定在明确的网络和端口上。第三，本机为 ARM Mac，很多网上现成 Hadoop 镜像只支持 amd64，直接使用会触发跨架构模拟，启动慢且不稳定。

因此，项目采用自建 ARM64 Hadoop 镜像，而不是使用 bde2020/hadoop-* 这类常见 amd64 镜像。镜像构建时只保留 Hadoop 运行所需的 Java、SSH、rsync、bash、procp、curl 和精简后的 /opt/hadoop，避免把旧容器里的 HBase、Hive、日志、HDFS 数据目录和 apt 缓存一起打包。早期如果使用 docker commit master，镜像会把运行时垃圾和历史数据也固化进去，体积可能膨胀到数十 GB；最终方案改为 Dockerfile 构建，使镜像来源更清楚，也更符合课程报告中的可复现实验要求。

Docker Compose 使用 yarn-lab profile 组织 Hadoop 与 Spark 服务^[9]。该 profile 下包含 NameNode、DataNode、SecondaryNameNode、ResourceManager、NodeManager、spark-client 和 Spark History Server。它们在同一个 Compose 网络中通过服务名互相访问，例如 Spark 作业通过 hdfs:// 路径访问 NameNode，通过 YARN ResourceManager 申请 executor 容器。项目还保留 local-lab profile 和本地 start-dev.sh，用于不启动完整 YARN 时进行 Flask/React 页面开发。

在端口规划上，本项目没有使用 Hadoop 默认宿主机端口，而是将 HDFS NameNode UI、YARN ResourceManager UI 和 Spark History UI 映射到 19870、18088 和 28080。这样做是因为宿主机上可能已经有旧的 master 容器占用 9870、8088 等端口。前后端开发则由 start-dev.sh 统一启动：Flask 后端默认监听 5051，Vite 前端监听 5173。脚本启动前会检查端口，如果对应端口已经被旧进程占用，会先终止旧进程再启动新服务，避免演示时手动查杀端口。

2.1.1 Docker、HDFS、YARN 与 Spark 的协同关系

答辩时可以把本项目的技术栈理解为三层协同。第一层是 Docker Compose，它负责把多个服务编排成一套可重复启动的本机实验集群；第二层是 Hadoop，其中 HDFS 负责存储输入样本、Parquet 结果和 Spark History 日志，YARN 负责分配 Spark executor 资源；第三层是业务系统，Spark 负责离线计算，Flask 负责读取计算结果并提供 API，React 负责展示业务指标和运行证据。

具体到容器角色，`enormous-data-hadoop-arm64:slim-hadoop` 是统一的 Hadoop 基础镜像，但它并不只启动成一个容器，而是由 `scripts/yarn_lab_role.sh` 根据参数分别启动为 NameNode、DataNode、SecondaryNameNode、ResourceManager 和 NodeManager。这样做的好处是镜像来源一致，运行角色清晰：NameNode 管理 HDFS 元数据，两个 DataNode 保存数据块，ResourceManager 负责全局资源调度，两个 NodeManager 承载 YARN container。Spark 相关容器则来自项目主 Dockerfile：`spark-client` 用于提交作业，`spark-history-server` 用于回放 event log，`web-yarn` 用于启动 Flask API。

表 2.1 Docker YARN 实验栈中的容器分工

容器或服务	所属层	作用
<code>yarn-namenode</code>	HDFS	保存 HDFS 目录树、文件元数据和 block 位置信息，对外提供 NameNode UI 与 RPC。
<code>yarn-datanode-1/2</code>	HDFS	保存实际数据块，模拟分布式文件系统多个存储节点。
<code>secondarynamenode</code>	HDFS	定期辅助 NameNode 做 checkpoint，降低元数据恢复压力。
<code>resourcemanager</code>	YARN	管理集群资源、队列和 application 生命周期，答辩时可在 UI 中查看 Spark 应用。
<code>nodemanager-1/2</code>	YARN	在节点侧启动 executor container，向 ResourceManager 汇报资源与运行状态。
<code>spark-client</code>	Spark	作为提交入口运行 <code>spark-submit</code> ，在 client mode 下也承载 Spark driver。
<code>spark-history-server</code>	Spark	从 <code>hdfs:///spark-history</code> 读取 event log，展示 stage、task、shuffle 和 executor 证据。
<code>web-yarn</code>	服务层	启动 Flask 后端，读取 <code>data/cache/*.json</code> 与 SQLite 作业记录，对外提供 <code>/api/v1</code> 。

一次完整的 YARN 模式运行可以分为六步。首先，Docker Compose 创建 `yarn-lab-net` 网络并启动 Hadoop/YARN/Spark/Flask 容器；其次，初始化脚本在

HDFS 中创建 `/user/course` 和 `/spark-history`；然后，样本 CSV 被上传到 HDFS 输入目录；接着，`spark-client` 或 `web-yarn` 调用 `submit_yarn_client.sh`，通过 `ResourceManager` 申请 `executor`；随后，`Spark executor` 从 HDFS 读取 CSV 或 Parquet，并把 `event log` 写回 `/spark-history`；最后，`Spark driver` 把前端需要的 JSON 指标写入 `data/cache`，`Flask API` 读取这些缓存供 `React` 展示。这个链路说明本项目不是让前端直接访问 HDFS，也不是让 `Flask` 在线计算百万级数据，而是采用“`Spark` 离线生产指标、`Flask` 在线读取缓存”的方式。

需要注意的是，本机 `Docker YARN` 是实验集群，不等同于真实多物理节点集群。所有容器仍然共享同一台 `Mac` 的 `CPU`、内存和磁盘，因此 `YARN` 模式主要证明分布式存储、资源调度和运行证据链路，而不保证在 1% 小样本上比 `local Spark` 更快。答辩演示可以使用 `local` 模式快速展示页面，再展示 `YARN` 相关 UI 和 `benchmark` 结果说明系统具备 `Hadoop/YARN` 扩展能力。

```
docker compose --profile yarn-lab up -d
docker compose --profile yarn-lab ps
docker compose --profile yarn-lab exec spark-client /app/scripts/init_yarn_lab.sh
docker compose --profile yarn-lab exec spark-client \
/app/scripts/submit_yarn_client.sh /app/configs/yarn-client.yaml yarn-demo
```

在本项目中，`Docker` 的作用不只是“打包运行环境”，更重要的是提供一个可控的课程实验边界：哪些服务启动、哪些端口开放、数据写到哪里、日志如何清理，都能在 `Compose` 配置和报告中明确说明。

2.2 HDFS 分布式存储

HDFS 是 `Hadoop` 的分布式文件系统，适合存储大文件和顺序扫描型数据。它由 `NameNode` 和 `DataNode` 组成：`NameNode` 保存文件目录、`block` 分布和权限等元数据，`DataNode` 保存实际数据块。大数据分析中，HDFS 的价值在于把输入数据从单机文件系统抽象为分布式存储路径，计算任务可以通过统一的 `hdfs://` 路径读取数据，并由集群负责数据块管理与容错^[10]。

本项目中，HDFS 主要承担三类数据的存储。第一类是输入数据，即从真实 `Kaggle Oct/Nov` 原始 CSV 中抽样得到的 1% 和 5% 用户级样本。样本上传到 `/user/course/ecommerce_behavior_user_sample_1pct/` 和 `/user/course/ecommerce_behavior_user_sample_5pct/`，`Spark` 作业通过配置文件读取这些路径。第二类是清洗后的中间结果，例如 `Parquet` 分区目录。相比反复扫描 CSV，`Parquet` 可以保存列类型、压缩信息和分区结构，更适合作为后续实验输入。第三类是 `Spark History event log`，保存在 `/spark-history`，供 `Spark History`

Server 回放作业执行过程。

HDFS 在项目中的使用遵循“原始层保留、计算层分区、日志层可追溯”的原则。原始 CSV 不直接覆盖；清洗后结果以 Parquet 目录保存；Spark event log 只保留最终实验需要的应用，删除早期 smoke、retry 和无效大日志。这样既能保留课程设计所需的 HDFS 截图和运行证据，也能控制本机 Docker 磁盘占用。

```
hdfs dfs -mkdir -p /user/course
hdfs dfs -mkdir -p /spark-history
hdfs dfs -put -f data/sample/ecommerce_user_sample_1pct.csv \
/user/course/ecommerce_behavior_user_sample_1pct/
```

从代码角度看，项目并不为本地文件和 HDFS 文件分别写两套读取逻辑。Spark 的 `spark.read.csv`、`spark.read.json` 和 `spark.read.parquet` 能同时处理本地路径和 HDFS 路径，因此项目只需要在 `configs/local.yaml`、`configs/hdfs.yaml`、`configs/yarn-client.yaml` 中切换 `data.input_path` 即可。这个设计降低了本地开发和 YARN 实验之间的迁移成本。

2.3 YARN 资源调度

YARN 是 Hadoop 生态中的资源管理层，核心组件包括 `ResourceManager`、`NodeManager` 和 `ApplicationMaster`^[11]。`ResourceManager` 负责全局资源调度，`NodeManager` 负责每台节点上的容器启动和资源上报，`ApplicationMaster` 负责某个具体应用的生命周期。在 Spark on YARN 模式下，Spark 不再只是在本地进程中运行，而是向 YARN 申请资源，由 YARN 分配 `executor` 容器执行任务^[12]。

本项目采用 YARN client mode 作为第一阶段实验模式。client mode 的特点是 driver 仍运行在提交作业的 `spark-client` 容器中，`executor` 由 YARN 调度到 `NodeManager` 上。这个选择与项目架构有关：Flask 和 React 前端仍读取 `spark-client` 可见的 `data/cache/*.json`，如果一开始就切到 cluster mode，driver 会运行在 `NodeManager` 容器中，输出缓存不一定自然落回 Flask 可读目录。因此，client mode 保留了本地缓存发布的便利性，同时让 `executor` 调度、YARN application 状态和 Spark History 日志进入课程实验范围。

YARN 在本项目中主要提供四类证据。第一，`ResourceManager` UI 能展示 `application id`、运行状态、开始时间和结束时间，证明 Spark 作业确实提交到 YARN。第二，`NodeManager` 参与执行 `executor` 容器，说明计算不只是 Flask 本地函数调用。第三，YARN application id 会写入 `run manifest` 和 `SQLite` 作业记录，前端 Ops 页可以直接展示。第四，失败或成功状态可以作为质量门禁的一部分，避免只看前端图表而忽略后端任务是否真正完成。

项目同时提供 `yarn-cluster.yaml` 和 `submit_yarn_cluster.sh` 作为后续扩展入口。`cluster mode` 会把 `driver` 交给 YARN 管理，更接近真实生产集群；但在当前课程设计中，`client mode` 更适合保证 Flask 缓存路径、前端展示和实验报告之间的一致性。

2.4 Spark SQL 与 DataFrame API

Spark 是本项目的数据计算核心。Spark 的 RDD 模型提供了分布式数据集、lineage 容错和内存复用能力^[3-4]，但项目业务代码主要使用 Spark SQL/DataFrame API，而不是直接操作 RDD。原因是电商行为分析天然结构化数据处理：字段类型固定，计算内容包括过滤、分组、聚合、窗口排序、join、去重和统计，这些操作更适合交给 DataFrame 和 Catalyst 优化器处理^[5]。

项目中所有原始事件会先进入 `read_events`，再经过 `clean_events` 生成 `cleaned_df`。清洗后 DataFrame 派生出三个关键中间结果：基础质量报告、会话事实表 `session_facts`、特征集市表。后续模块如转化漏斗、行为路径、推荐、异常检测、生命周期、实验分流都复用这些中间结果。这样避免了每个模块重复解析时间、重复处理空值、重复去重，也保证同一字段在不同页面中的含义一致。

Spark SQL 在各模块中的应用非常具体。例如：

- 数据清洗使用 `withColumn`、`to_timestamp`、`coalesce`、`filter` 和 `dropDuplicates`。
- 基础看板使用 `groupBy`、`count`、`sum`、`avg` 和 `countDistinct`。
- 转化分析按 `user_session` 聚合首次浏览、首次加购、首次购买时间，再计算合法路径。
- 归因模块用 `window function` 计算触点在一次购买前的 `first-touch`、`last-touch`、`linear` 和 `time-decay credit`。
- 推荐模块用窗口函数按会话排序，保留每个会话 `top-k` 商品。
- 异常检测使用 `percentile_approx` 计算 `median` 和 `MAD`，再得到 `robust z-score`。

```
cleaned_df = clean_events(raw_df).persist(StorageLevel.MEMORY_AND_DISK)
session_facts = build_session_facts(cleaned_df).persist(StorageLevel.MEMORY_AND_DISK)
metrics = {
    **build_metrics(cleaned_df, quality, top_n),
    **build_conversion_metrics(cleaned_df, session_facts, top_n),
}
```

在本项目中，Spark 不是单独存在的计算脚本，而是整个系统的生产者：它负

责把原始行为日志转换为前端能直接展示的结构化 JSON、Parquet 和运行 manifest。

2.5 Spark AQE 与执行计划优化

Spark SQL 的 Adaptive Query Execution (AQE) 能够在运行时根据真实 shuffle 统计信息调整执行计划，例如合并小 shuffle 分区、处理倾斜 join 和选择更合适的 join 策略^[13]。这对本项目很重要，因为电商行为数据存在天然倾斜：少数热门类目、热门品牌和热门商品会拥有远高于普通商品的浏览和购买次数；同时推荐、关联图谱、归因等模块会产生较多 join 和 window 操作。

项目在 `configs/yarn-client.yaml` 中启用 AQE、skew join 和合理的 shuffle partitions。1% 样本 benchmark 中，YARN-only CSV 用时明显高于 YARN + AQE 组，说明只把作业交给 YARN 并不会自动优化 SQL 执行计划；AQE 介入后，shuffle 分区和倾斜处理更合理，整体耗时明显下降。这个实验结果也是报告第 4 章“YARN 不是性能优化本身”的证据来源。

除了 AQE，本项目还做了一个与 Spark History 相关的配置：将 `spark.sql.maxPlanStringLength` 限制为 8192。原因是 Spark History Server 会回放 event log，如果 SQL plan 字符串过长，History API 可能在解析 stages 或 jobs 时返回 500。限制 plan 字符串不会改变计算结果，但能降低 event log 体积和 History Server 解析风险。这属于工程稳定性优化，直接服务于前端 Ops 页和报告截图。

```
spark.sql.adaptive.enabled: true
spark.sql.adaptive.skewJoin.enabled: true
spark.sql.shuffle.partitions: 64
spark.sql.maxPlanStringLength: 8192
spark.eventLog.enabled: true
spark.eventLog.dir: hdfs:///spark-history
```

2.6 Parquet 与数据布局

CSV 是本项目的原始输入格式，因为 Kaggle 数据集本身以 CSV 提供，字段直观，便于检查和抽样。但 CSV 不适合作为反复分析的中间层：它没有列式编码，不保存强类型信息，不能有效做列裁剪和谓词下推，每次读取都需要重新解析文本。随着数据量从小样本扩大到 1%、5% 用户级样本，重复扫描 CSV 会成为明显开销。

Parquet 是列式存储格式，适合 Spark SQL 的分析型读取，支持压缩、列裁剪和谓词下推^[14]。本项目采用“CSV 原始层 + Parquet 清洗层”的布局：原始 Kaggle

CSV 保留在 `data/raw/kaggle/ecommerce_behavior`；用户级样本上传 HDFS 作为实验输入；Spark 清洗完成后，将标准化事件写成 HDFS Parquet 分区目录；Parquet 对照组再从该目录读取数据，跳过部分 CSV 解析和清洗成本。

在报告实验中，Parquet 的意义不只是“更快”。它还提供了更清楚的数据分层：CSV 代表外部原始数据，Parquet 代表经过清洗、类型统一和质量校验后的分析层。后续如果要把 `cluster mode` 设为默认路径，或者把 Flask 从本地 JSON cache 改为读取 HDFS 产物，Parquet 清洗层就是更合适的接口。

2.7 内存优化与数据倾斜控制

Spark 作业出现 OOM 通常有两类原因：一类是 `executor` 端 `shuffle`、`join` 或聚合压力过大；另一类是 `driver` 端把太多结果 `collect()` 到本地。本项目最初的小样本演示可以把明细直接收集成 JSON，但当数据量扩大后，推荐明细、`cohort` 明细、`forecasting` 历史和 `experimentation` 分流行数都会增长，`driver` 端内存风险很高。

因此，项目把优化分成执行层和算法层两部分。执行层优化包括启用 AQE、设置合理 `shuffle partitions`、使用 `MEMORY_AND_DISK` 缓存、模块结束后 `unpersist()`、限制 Spark History plan 字符串。算法层优化包括：推荐模块只 `collect preview limit` 行，质量指标改由 Spark 聚合计算；`forecasting` 模块只把全站和 `top` 类目的有限历史带到 `driver`；关联图谱限制 `top nodes` 和单会话最大商品数；`experimentation` 只返回前端展示所需 `preview`，完整分流结果保留为 `DataFrame`。

这些做法与 Starfish、SkewTune 和 Themis 的思想一致。Starfish 强调通过运行画像调优，而不是凭经验猜参数^[6]；SkewTune 指出数据倾斜和长尾 task 会拖慢整个作业^[7]；Themis 说明系统不应盲目追求全内存，合理 `spill` 和磁盘中间结果反而更稳定^[8]。本项目没有重新实现这些研究系统，而是在 Spark 工程中落地为 AQE、Parquet、预聚合、限流、质量报告和 `benchmark` 证据。

2.8 Flask API 服务层

Flask 在项目中承担服务层角色，负责把 Spark 产出的 JSON 缓存、作业记录和 `benchmark` 证据封装成前端可调用接口。后端以 `Blueprint` 组织路由，统一挂载在 `/api/v1` 下。每个 API 返回统一的 `ApiEnvelope` 结构，包含 `code`、`message`、`data` 和 `meta.request_id`。这种结构让前端无需为每个接口单独判断错误格式，也便于 Ops 页展示失败信息。

Flask 服务并不在用户访问页面时同步跑完整 Spark 作业。常规页面读

取 `data/cache/*.json` 中的指标缓存，例如 `summary.json`、`daily_events.json`、`recommendation_items.json`。当用户触发刷新时，后端通过 `JobService` 创建作业记录，再调用 `SparkPipelineRunner` 提交 `Spark` 作业。作业完成后，系统读取 `run manifest`，把输入快照、输出文件、质量报告、`Spark application id` 和 `History` 指标写入 `SQLite` 作业表。

这种设计把“计算”和“接口响应”拆开了。前端访问页面时得到的是轻量 `JSON`，不会等待 `Spark` 长任务；`Spark` 作业的状态、失败阶段和质量门禁由作业 `API` 查询。对于课程设计来说，这比在 `Flask` 路由中直接调用 `Spark` 计算更稳定，也更容易解释系统架构。

```
@api_bp.get("/summary")
def summary():
    return api_ok(cache().load_metric("summary"))

@api_bp.post("/refresh")
def refresh():
    job_record = job_service().enqueue_refresh()
    return api_ok({"status": "queued", "job_id": job_record.job_id}, "refresh queued"), 202
```

2.9 React、ECharts 与前端可视化

前端使用 `React + Vite` 构建，`ECharts` 负责图表渲染，`TanStack Query` 负责接口请求和缓存。`React` 的作用是把业务页面拆成可维护的组件，例如指标卡片、图表面板、数据表格、布局导航和错误状态；`ECharts` 的作用是将 `Spark` 聚合结果展示为折线图、柱状图、饼图、漏斗图和关系图；`TanStack Query` 则负责加载状态、错误状态和重复请求缓存。

项目页面按照分析主题组织，而不是只做一个简单首页。路由包括 `Dashboard`、`Behavior`、`Conversion`、`Journey`、`Forecasting`、`Affinity`、`Cohorts`、`Portfolio`、`Cart Recovery`、`Attribution`、`Optimization`、`Recommendations`、`Anomaly`、`Lifecycle`、`Experiments`、`Feature Mart`、`Quality`、`Table` 和 `Ops`。每个页面对应后端若干 `API`，例如转化页读取 `/conversion/funnel`、`/conversion/daily` 和 `/conversion/products`；推荐页读取 `/recommendations/items`、`/recommendations/quality` 和 `/recommendations/alerts`。

前端可视化不是直接处理原始大数据，而是展示 `Spark` 预聚合后的指标。这样做有两个好处：第一，浏览器不需要加载百万级原始行，页面响应更快；第二，图表口径由 `Spark` 统一计算，避免前端重复实现统计逻辑。对于表格页面，后端提供分页和筛选参数，前端只请求当前页所需数据。

2.10 Ops 证据面板与可追溯性

Ops 页面是本项目区别于普通可视化大作业的重要部分。普通看板只回答“业务指标是什么”，Ops 页面回答“这些指标从哪里来、Spark 是否跑完、输入数据是什么、质量是否通过、benchmark 结果如何”。它读取的不是业务分析接口，而是 `/api/v1/ops/evidence`、`/api/v1/job`、`/api/v1/jobs`、`/api/v1/jobs/<job_id>/lineage` 等运行证据接口。

后端的 `BenchmarkEvidenceService` 会读取正式 benchmark 目录，只返回精简字段：样本、对照组、耗时、吞吐、YARN application id、质量状态、HDFS 输入路径和清理策略。它不会把 Spark stderr 或完整 event log 返回给浏览器，避免长日志造成前端卡顿。Spark History 指标则通过 History API 或 event log 解析脚本补采集，最后形成可展示的 shuffle、spill、failed task 和 retried task 证据。

在课程报告中，Ops 面板承担“证据汇总”的角色。它把 HDFS 输入、YARN application、Spark History、质量门禁、benchmark 和清理策略放在一个页面，答辩时不需要在多个终端窗口之间来回切换。这个设计也让报告中的实验结论更可信：不是只写“系统可以运行”，而是能展示每次运行的输入、输出、作业状态和指标结果。

2.11 SQLite 作业记录与缓存文件

项目使用 SQLite 保存刷新作业的元信息，使用 `data/cache/*.json` 保存前端展示指标。SQLite 适合本课程设计的原因是它不需要单独部署数据库服务，文件可随项目一起备份和检查，同时足够保存 job id、run id、状态、失败阶段、Spark application id、质量报告和 artifact 信息。对于本机演示和课程验收来说，SQLite 比引入 MySQL 或 PostgreSQL 更轻量。

JSON 缓存则是 Spark 与 Flask/React 之间的交付格式。Spark 作业完成后，`write_metric_files` 将 summary、event distribution、daily trend、recommendations、quality、ops evidence 等指标写入 `data/cache`。Flask 的 `MetricCache` 只负责读取这些文件并做分页、过滤或字段裁剪。这样一来，Spark 作业可以独立运行，Flask 服务也可以独立重启；只要缓存存在，前端页面就能展示最近一次成功运行的结果。

需要注意的是，本项目并没有把所有明细都写成 JSON。大规模明细适合放在 Parquet 或 Spark DataFrame 中，JSON 只保存 summary、quality、manifest 和 preview。这个取舍贯穿整个系统：前端要的是可解释、可交互、响应快的指标，不是把分布式计算结果原样塞进浏览器。

第3章 需求分析（设计目标）

3.1 总体目标

系统的总体目标是构建一套可运行、可复现、可对比的电商行为大数据分析实验平台。平台既要能处理真实 Kaggle Oct/Nov 行为数据，又要适合本机 Docker Desktop 环境，不能为了追求形式上的“集群”而频繁 OOM 或占满磁盘。最终交付应包含四类成果：第一，ARM64 Docker Hadoop YARN 实验集群；第二，Spark 分析 pipeline 与内存优化实现；第三，1% 和 5% benchmark 对照结果；第四，前端 Ops 页面和 LaTeX 报告中的实验图表。

3.2 功能需求

3.2.1 数据处理功能

数据处理链路需要支持从原始 CSV 到清洗缓存的完整闭环。原始数据来自 Kaggle 电商行为数据集，字段包含 event_time、event_type、product_id、category_id、category_code、brand、price、user_id 和 user_session。系统需要完成字段类型转换、异常价格过滤、重复事件处理、会话事实表生成、日趋势、转化漏斗、商品推荐、关联图谱、异常信号、cohort、forecasting、experimentation 和 feature mart 等分析模块。

3.2.2 集群运行功能

系统需要通过 Docker Compose 启动 Hadoop YARN 实验集群，并支持 Spark submit 在 YARN client mode 下运行。NameNode、ResourceManager 和 Spark History Server 必须提供可访问 UI，便于答辩时展示 HDFS 文件、YARN application 和 Spark event log。由于已有 master 容器占用默认端口，新集群必须使用偏移端口，避免端口冲突。

3.2.3 运维展示功能

前端 Ops 页面需要展示当前 Spark 作业状态、质量门禁、HDFS 输入快照、Spark application id、Spark History 指标状态、最近运行记录和 benchmark 证据。新增 benchmark 展示应覆盖 1% 五组、5% 两组实验结果，并突出 YARN-only、AQE、算法优化和 Parquet 输入之间的差异。

3.3 非功能需求

3.3.1 架构兼容性

本项目必须兼容 ARM Mac。所有 Hadoop/Spark 相关镜像必须使用 ARM64 架构，不能下载 amd64 镜像，也不能依赖 linux/amd64 平台模拟。最终镜像不采用 `docker commit master` 这种粗暴方式，而是构建只包含 Java、Hadoop、必要配置和启动脚本的精简镜像。

3.3.2 内存稳定性

系统目标不是承诺任意数据规模都不会 OOM，而是在本机资源范围内显著降低 OOM 风险，并通过实验给出证据。driver 端不得收集完整明细；大输出应落地为 Parquet；JSON 只保留 summary、quality、manifest 和 preview；模块缓存必须使用 `MEMORY_AND_DISK`，并在模块结束后释放。对于关联分析、推荐和异常检测，应设置候选上界，避免二次级组合膨胀。

3.3.3 证据可追溯性

每次运行应保留 run manifest、配置 hash、input snapshot、quality report、Spark application id 和 output artifact row counts。Benchmark 结果应保存到 data/benchmarks，报告图表保存到 报告/figures。前端展示的数据不应依赖人工复制终端输出，而应由 API 自动读取正式 benchmark 文件。

3.4 实验设计

实验规模分为 1% 用户级样本和 5% 用户级样本。1% 样本用于五组对照，5% 样本用于压力验证。对照组设计如表3.1所示。

3.5 验收指标

系统验收指标包括工程指标和实验指标两类。工程指标要求 `docker compose --profile yarn-lab up -d` 后，NameNode、ResourceManager、NodeManager、Spark client 和 Spark History Server 能够启动；`start-dev.sh` 能在 5051/5173 启动后端与前端；HDFS 能保存样本数据；Spark submit 能产生 SUCCEEDED application；前端 Ops 页能显示 benchmark、HDFS 输入、YARN app id 和质量状态。

实验指标要求记录 elapsed seconds、cleaned rows/sec、input rows、output rows、Spark application status、driver peak memory、shuffle/spill 状态、failed/retried task count、

表 3.1 实验对照组设计

对照组	实验目的
Baseline local CSV	观察单机 local Spark 在 1% 样本上的基线耗时和 driver 内存。
YARN-only CSV	只切换到 YARN client mode，不做算法优化，验证“只加 YARN”是否有效。
YARN + AQE CSV	启用 AQE、skew join 和合理 shuffle partitions，验证执行计划优化收益。
YARN + algorithm CSV	在 AQE 基础上限制 collect、关联组合和 preview 输出，验证算法层优化收益。
YARN + Parquet	使用清洗后的 Parquet 输入，验证列式数据布局收益。

output artifact row counts 和 quality gate status。其中最关键的结论阈值如下：

1. YARN application 最终状态为 SUCCEEDED。
2. driver 不再因为 full result collect() 或大 JSON payload OOM。
3. recommendation 明细收集行数固定为 preview limit，质量统计由 Spark 聚合完成。
4. pair_base_rows / cleaned_rows 保持在可解释范围内，不能出现未受控的二次级组合膨胀。
5. 5% 样本相较 1% 样本运行时间增长应接近线性，不能出现失控长尾。
6. Spark History Server 能查看最终实验 event log；旧 event log 若触发 JSON 限制，应清理或用限制 plan string 后的日志替代。

3.6 数据清理需求

旧数据集和旧实验日志会快速占用 Docker 与 HDFS 空间，因此系统需要及时清理冗余产物。清理策略是：保留原始 Kaggle Oct/Nov CSV；保留 1% 与 5% 本地样本；保留正式 benchmark 目录；保留最终 6 个 Spark History event log；删除早期 smoke、retry、history recheck benchmark 目录和旧的大 History 日志。清理后的存储快照见第 4 章图4.4。

第4章 系统设计与实现（实现步骤截图、环境安装、代码实现）

4.1 总体架构

系统采用“数据层、计算层、服务层、展示层”四层结构。数据层包括本地 Kaggle 原始 CSV、本地 1%/5% 样本、HDFS CSV 输入和 HDFS Parquet 清洗层；计算层包括 Spark SQL pipeline、AQE、推荐/关联/异常等算法模块和 Spark History event log；服务层包括 Flask API、SQLite 作业记录、run manifest 与 benchmark evidence 服务；展示层包括 React 前端业务页面、质量页和 Ops 页。

图4.1展示了 Docker YARN 实验集群拓扑。该拓扑保留了 Hadoop 课程设计所需的 NameNode、DataNode、ResourceManager、NodeManager 和 Spark History Server，同时增加 spark-client 作为提交入口。前端与后端按 `start-dev.sh` 启动：Flask 后端为 5051，Vite 前端为 5173；HDFS UI、YARN UI 和 Spark History UI 分别映射到 19870、18088 和 28080，避免与已有 master 容器冲突。

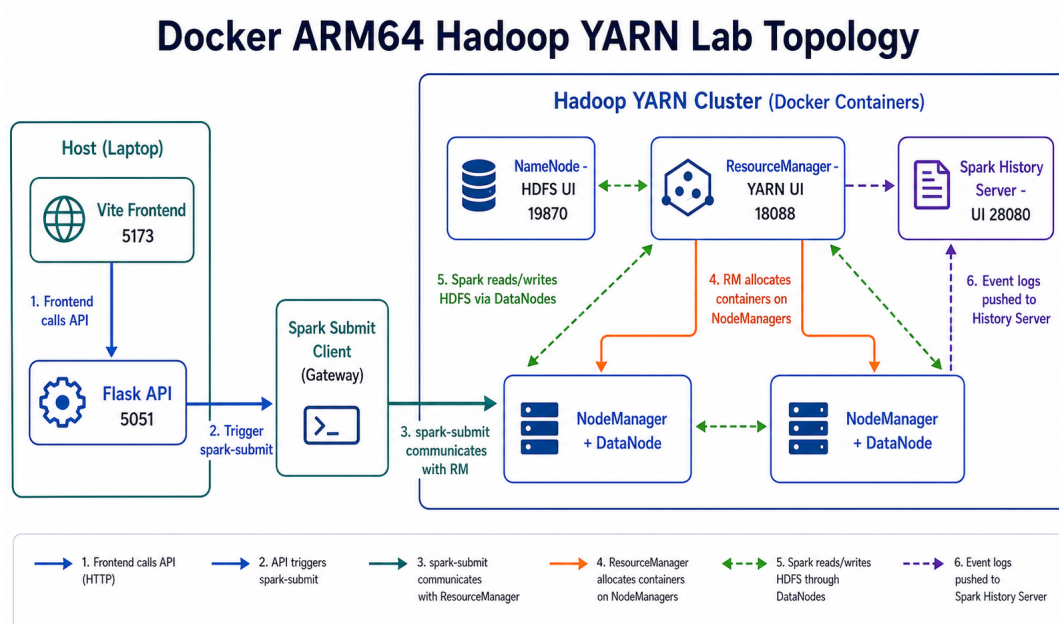


图 4.1 Docker ARM64 Hadoop YARN 实验集群拓扑（由 chat-gpt-image-2 生成）

4.2 环境安装与启动步骤

4.2.1 ARM64 Hadoop 镜像构建

本项目避免使用 amd64 Hadoop 镜像，也避免把整个 master 容器 commit 成大镜像。最终方案是自建精简 ARM64 Hadoop 镜像，只包含运行 Hadoop 所需的 Java、Hadoop 文件、配置和启动脚本。核心思路如下：

```
FROM ubuntu:24.04

RUN apt-get update && apt-get install -y \
    openjdk-8-jdk-headless bash ssh rsync procps curl \
    && rm -rf /var/lib/apt/lists/*

COPY hadoop /opt/hadoop

ENV HADOOP_HOME=/opt/hadoop
ENV HADOOP_CONF_DIR=/opt/hadoop/etc/hadoop
ENV PATH=/opt/hadoop/bin:/opt/hadoop/sbin:$PATH
```

与 docker commit master 相比，该方法不会复制 HDFS 数据目录、日志、Hive、HBase 和临时文件，因此镜像体积更可控，也符合 ARM Mac 的架构约束。

4.2.2 Docker Compose 启动

集群通过 yarn-lab profile 启动。常用命令如下：

```
docker compose --profile yarn-lab up -d
docker compose --profile yarn-lab ps
docker compose --profile yarn-lab config --quiet
```

端口规划如表4.1所示。Hadoop UI 端口均为偏移端口，原因是用户原有 master 容器已经占用 Hadoop 默认 UI 端口；前后端访问地址则以 start-dev.sh 为准。

本项目保留两套演示入口。答辩现场如果重点展示前端页面和业务分析结果，推荐使用本地开发模式，因为它复用宿主机 Python 虚拟环境和 Vite，启动更快，端口也与前端代理一致：

```
.venv/bin/python -m spark_jobs.main --config configs/local.yaml
./start-dev.sh
```

如果需要展示 Hadoop/YARN 技术栈，则启动 yarn-lab，再打开 HDFS、YARN 和 Spark History UI。该模式下 web-yarn 的宿主机 API 地址为 http://127.0.0.1:5050/api/v1，前端可以通过新增的 start-yarn-dev.sh 指向该地址：

```
./start-yarn-dev.sh
```

表 4.1 实验环境端口规划

服务	宿主机端口	说明
Flask API	5051	<code>start-dev.sh</code> 启动的后端接口, 前端通过该地址访问 <code>/api/v1</code> 。
Vite 前端	5173	<code>start-dev.sh</code> 启动的正确前端访问地址, 用于打开 <code>/ops</code> 。
YARN Flask API	5050	<code>web-yarn</code> 容器的宿主机端口映射, 内部仍监听 5051。
NameNode UI	19870	查看 HDFS 文件、block 和 DataNode 状态。
ResourceManager UI	18088	查看 YARN application、队列和运行状态。
Spark History Server	28080	查看 Spark event log 和 stage/job 信息。

两套模式的区别不在前端页面, 而在数据刷新方式。local 模式使用 `configs/local.yaml`, Spark 直接读取本地 `data/sample/ecommerce_user_sample_1pct.csv` 并写入 `data/cache`; YARN 模式使用 `configs/yarn-client.yaml`, Spark 从 `hdfs:///user/course/ecommerce_behavior_user_sample_1pct/*.csv` 读取输入, 并在 ResourceManager 中生成 YARN application。无论哪种模式, 前端最终读取的都是 Flask API 返回的 JSON 指标, 因此页面展示逻辑保持一致。

4.2.3 HDFS 目录准备

启动后需要创建实验目录, 并上传样本数据。正式样本来自真实 Oct/Nov 原始 CSV 的用户级采样, 不再使用 87.4KB smoke 文件作为论文结论依据。

```
docker compose --profile yarn-lab exec spark-client \
  /app/scripts/init_yarn_lab.sh
PERCENTS="1 5" ./scripts/prepare_yarn_samples.sh
```

`init_yarn_lab.sh` 的作用是等待 HDFS 可用、创建 `/user/course` 与 `/spark-history`, 并按参数上传样本文件。正式实验中, `prepare_yarn_samples.sh` 会从本地用户级样本准备 1% 和 5% 输入, 再上传到 HDFS 对应目录。这样 HDFS 中的数据来源可以追溯到本地样本文件和原始 Kaggle CSV, 而不是手工复制的不明来源文件。

清理后的 HDFS 关键路径包括 1%/5% CSV 输入、YARN-only/AQE/algorithm/Parquet 的处理输出, 以及 6 个最终 Spark History event log。旧的 `application_178098*` 和 `application_1780991452919_0001` 到 `0005` 大

日志已经清理，避免继续触发 Spark 3.5.5 History Server 的 JSON 长字符串限制。

4.3 Spark 提交链路实现

4.3.1 YARN client 配置

第一阶段采用 YARN client mode。这样 driver 运行在 spark-client 容器，Flask 仍可读取本地 data/cache，前端也能继续使用原有缓存文件。核心配置包括：

```
spark:
  master: yarn
  deployMode: client
  appName: ecommerce-yarn-lab
  config:
    spark.sql.adaptive.enabled: true
    spark.sql.adaptive.skewJoin.enabled: true
    spark.sql.shuffle.partitions: 64
    spark.sql.maxPlanStringLength: 8192
    spark.eventLog.enabled: true
    spark.eventLog.dir: hdfs:///spark-history
```

spark.sql.maxPlanStringLength 是本次修复的重要配置^[15]。旧 5% event log 在 Spark History Server 的 /stages API 上会触发 JSON 长字符串限制，限制 plan string 后重新运行的最终 event log 体积明显降低，便于长期保留。

4.3.2 提交脚本

提交前先将 spark_jobs 打包为 zip，再通过 spark-submit 提交到 YARN：

```
docker compose --profile yarn-lab exec spark-client \
/app/scripts/submit_yarn_client.sh /app/configs/yarn-client.yaml yarn-demo
```

脚本内部会设置 Hadoop 配置目录、Spark driver host、Python 3.11 解释器路径和 event log 参数，然后调用 spark-submit --master yarn --deploy-mode client。如果用户从前端点击“刷新 Spark”，Flask 的 /api/v1/refresh 会创建 SQLite 作业记录，再由 SparkPipelineRunner 调用 SPARK_SUBMIT_SCRIPT 指定的提交脚本。运行完成后，SparkPipelineRunner 从 stdout 中解析 Spark run manifest 路径，读取 manifest 并写入 SQLite 作业表。为了避免后端消息过长，runner 只保留 stdout/stderr tail，完整日志写入 data/cache/run_logs。

需要理解的是，前端展示的业务指标并不是在每次页面刷新时实时计算。Spark 作业成功后会覆盖 data/cache/*.json；如果当前 YARN 任务仍处于 running，页面仍会显示最近一次成功缓存中的业务指标，同时 Ops 页显示当前任务的 running 状态。这种“旧结果可用、新任务后台刷新”的模式能够保证答辩页面稳定，不会因

为 Spark 长任务未完成而空白。

4.3.3 YARN cluster mode 入口

在第一阶段，系统默认使用 YARN client mode，因为 Flask 与 React 仍读取 spark-client 容器中的 data/cache。同时项目新增了 configs/yarn-cluster.yaml 与 scripts/submit_yarn_cluster.sh，作为第二阶段 cluster mode 入口。cluster mode 会把 driver 交给 NodeManager 运行，并通过 --files 分发配置文件，通过 spark.yarn.appMasterEnv.PYSPARK_PYTHON 和 spark.executorEnv.PYSPARK_PYTHON 固定 Python 3.11，避免 driver 与 executor Python 小版本不一致。由于 cluster mode 下本地 JSON cache 不再天然落在 Flask 可读目录，后续若要把 cluster mode 设为默认刷新路径，需要继续实现 HDFS 输出读取或缓存同步。

4.4 核心算法与内存优化实现

4.4.1 禁止 full collect

早期推荐模块会把 recommendation_features.collect() 的完整结果带回 driver，再生成大 JSON。这在小样本下能运行，但数据量扩大后会直接增加 driver OOM 风险。优化后，summary、coverage、fallback、duplicate 和 confidence 等质量指标全部使用 Spark 聚合完成；前端展示只收集 preview limit 行数；完整明细写入 Parquet。

```
preview_rows = (
    recommendation_features
    .orderBy(F.desc("score"))
    .limit(preview_limit)
    .collect()
)

quality = recommendation_features.agg(
    F.count("*").alias("candidate_rows"),
    F.avg("confidence").alias("avg_confidence"),
    F.sum(F.col("is_fallback").cast("int")).alias("fallback_rows"),
)
```

4.4.2 WindowExec 警告优化

Spark 日志中曾出现大量 WindowExec: No Partition Defined 警告。这类警告表示窗口函数没有分区键，Spark 会把数据移动到单分区处理，小样本时只是警告，全量时会变成瓶颈。优化后，ranking 逻辑统一通过 helper 添加全局 rank 或业

务分区 rank，避免模块中散落无分区 `Window.orderBy()`。

4.4.3 关联分析组合膨胀控制

商品关联图谱最容易产生二次级组合膨胀。若某个 session 中商品数过多，直接两两配对会产生 $O(n^2)$ 行。系统加入 `top_nodes`、`max_pair_products_per_session` 和 pair explosion 指标。质量报告记录 `eligible_session_count`、`pair_base_rows` 和 `pair_rows_per_input_row`，用于判断数据规模增长是否仍受控。

4.4.4 Parquet 清洗层

CSV 原始层只作为输入层。清洗后的事件表写入 HDFS Parquet，并按日期字段分区。Parquet 输入对照组直接读取清洗目录，跳过 CSV schema 解析和清洗阶段的部分开销。图4.2展示了从原始 CSV 到前端 Ops 证据的处理流程。

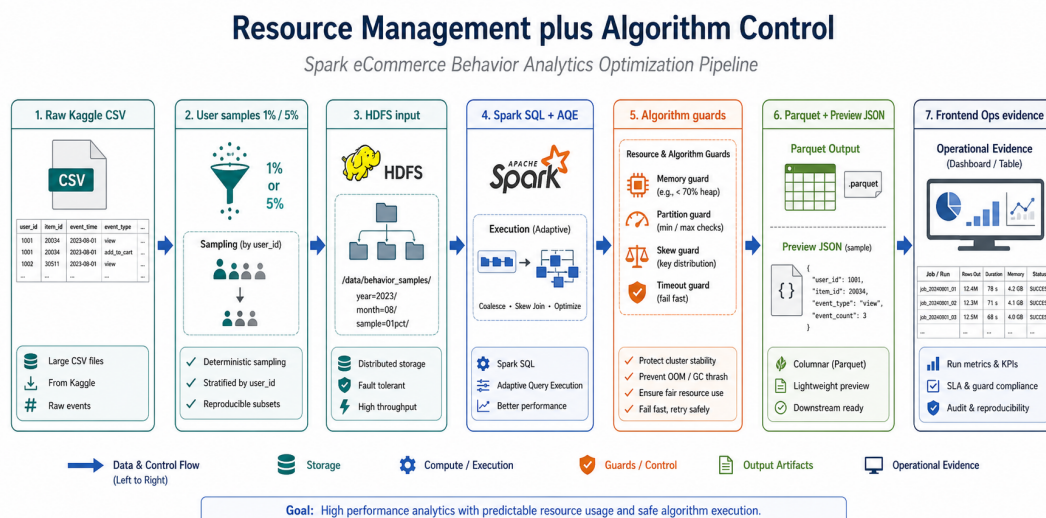


图 4.2 Spark 数据处理与优化流程（由 chat-gpt-image-2 生成）

4.5 核心业务模块数据计算方法

本系统的计算入口是 `spark_jobs/main.py` 中的 `run_job`。一次运行会先读取原始事件表，再生成清洗事件表 `cleaned_df`、会话事实表 `session_facts` 和特征集市表，随后各业务模块复用这些中间结果。这样做的原因是：事件清洗、去重、日期转换和基础聚合只做一次；转化、推荐、异常、生命周期等模块使用统一口径，避免同一个指标在不同页面出现不一致。

4.5.1 基础清洗与元数据口径

原始数据读取时使用固定 schema，而不是依赖 Spark 自动推断。CSV、JSON、TXT 与 Parquet 入口最终都映射为同一组字段：event_time、event_type、product_id、category_id、category_code、brand、price、user_id 和 user_session。清洗阶段完成四件事：

1. 将 event_time 转为 event_timestamp，同时派生 event_date 与 event_hour。
2. 将空品牌、空类目和空会话统一填充为 unknown，把 category_code 拆出一级、二级类目。
3. 过滤非法事件类型、空时间、空商品、空用户以及价格小于 0 或大于 100000 的异常行。
4. 按 event_time + event_type + product_id + user_id + user_session 去重。

质量报告中的 raw_rows 为原始行数，cleaned_rows 为清洗后行数，removed_rows = raw_rows - cleaned_rows，removed_ratio = removed_rows / raw_rows。重复行、异常价格行、缺失品牌行、缺失必要主键行分别由 Spark filter 或 dropDuplicates 统计。该口径对应前端质量页和 Ops 页的基础数据质量指标。

清洗阶段核心代码口径

```

1 cleaned = (
2     normalized
3     .filter(F.col("event_type").isin(EVENT_TYPES))
4     .filter(F.col("event_timestamp").isNotNull())
5     .filter(F.col("product_id").isNotNull())
6     .filter(F.col("user_id").isNotNull())
7     .filter(F.col("price").isNull() | ((F.col("price") >= 0) & (F.col("price") <= 100000)
8         ↪ ))
9     .dropDuplicates(["event_time", "event_type", "product_id", "user_id", "user_session"
10         ↪ ])
11 )
12
13 quality = {
14     "raw_rows": raw_df.count(),
15     "cleaned_rows": cleaned.count(),
16     "duplicate_rows": raw_df.count() - raw_df.dropDuplicates(event_key_cols).count(),
17 }

```

4.5.2 模块计算总览

表4.2汇总了每个页面模块的数据来源、主要计算方法和输出指标。所有 preview 类 JSON 都只保留前端展示需要的有界行数；完整明细或大中间结果保留为

Spark DataFrame 或 Parquet，避免 driver 端收集全量明细。

表 4.2 业务模块计算口径总览

模块	输入表	计算方法	输出指标
基础看板	cleaned_df、quality	按事件类型、日期、类目、品牌聚合；购买事件金额求和；用户和会话去重计数。	总行数、有效行数、销售额、用户数、会话数、行为分布、日趋势、排行。
转化分析	session_facts、cleaned_df	以会话为单位寻找首次浏览、首次加购、首次购买时间，要求时间顺序合法；再统计 view→cart→purchase 漏斗。	会话漏斗、日转化、商品转化率、平均下单延迟、客单价。
行为路径	cleaned_df	按 user_session 收集并排序事件序列，截取前 N 个事件形成 path signature；相邻事件数组 zip 后 explode 为转移边。	高频路径、事件转移、退出事件、购买路径、路径转化率。
特征集市	raw_df、cleaned_df	生成 SHA-256 event key 去重；按日-商品、日-用户、日-类目三种粒度聚合行为、金额和偏好。	商品/用户/类目日特征、重复率、隔离率、分区数、新鲜度。
需求预测	cleaned_df	先生成全站和类目日需求序列，再选择 GMV 最高的 top entities；用滚动窗口均值和近周期变化率生成未来 7 日预测。	预测序列、实体预测摘要、回测 WAPE、风险预警。
关联图谱	cleaned_df	将会话内商品行为压缩为 product-session 表；只保留 top 商品，并限制单 session 商品数；两两配对后计算 support、confidence、lift、Jaccard。	节点、边、社群、交叉销售机会、组合膨胀质量指标。
Cohort 留存	购买事件	找到用户首次购买月份作为 cohort；按购买月份计算 period index；统计活跃、复购、收入和累计收入曲线。	留存矩阵、复购间隔、价值曲线、cohort 风险分群。
商品组合	cleaned_df	按类目、品牌、价格带聚合浏览/加购/购买/收入；计算收入占比、购买占比和商品 HHI 贡献。	类目结构、品牌结构、价格带、商品集中度、组合机会。
购物车挽回	cleaned_df	以会话-商品为粒度提取加购、移除、购买；购买晚于加购则视为 recovered，否则 abandoned。	类目挽回率、商品放弃率、放弃金额、挽回队列。

模块	输入表	计算方法	输出指标
收入归因	cleaned_df	每次购买向前关联同会话触点；计算 first-touch、last-touch、linear 和 time-decay 四种收入 credit。	归因模型汇总、商品/品牌/类目贡献、路径模式、助攻机会。
运营优化	cleaned_df、全局转化率	构造商品候选，使用 Wilson 下界与贝叶斯收缩得到置信购买率；在预算、坑位、类目和品牌约束下求最大化收益计划。	候选商品、优化计划、目标值、求解状态、质量状态。
推荐	cleaned_df、优化计划	计算商品转化特征与商品分；提取目标会话偏好类目；同类目个性化候选优先，兜底商品补齐，按 score 排 top-k。	推荐列表、覆盖率、fallback 率、平均置信度、质量告警。
异常检测	feature mart	将类目和 top 商品日指标拉平成监控信号；用 median 与 MAD 计算 robust z-score，并按阈值分级。	异常告警、时间线、监控规则、雷达状态。
生命周期	日用户特征、日类目特征	按用户聚合活跃天数、收入、购买天数、最近活跃时间和偏好类目；用 RFM 与行为规则分层。	用户生命周期分布、风险队列、类目偏好、行动建议。
实验分流	生命周期用户、推荐特征、优化计划	构造实验目录；对 eligible 用户按 hash(user, experiment) 稳定分桶；统计 treatment/control 平衡性和预期增量 GMV。	实验目录、分流明细、分层准备度、guardrail。
Ops 证据	run manifest、SQLite、benchmark 文件	汇总运行状态、input snapshot、artifact row counts、Spark application id、History 指标和 benchmark 对照结果。	作业状态、质量门禁、HDFS 输入、实验结果、清理策略。

4.5.3 基础看板与排行模块

基础看板是所有可视化页面的入口。其核心指标来自 spark_jobs/aggregations.py。行为分布按 event_type 分组计数；每日事件趋势按 event_date 或单日场景下的 10 分钟桶分组计数；每日销售额只统计 event_type = purchase 的价格求和；类目排行按一级类目事件数排序；品牌排行只统计购买事件的订单数和销售额。

关键公式如下：

$$\text{total_sales} = \sum_{\text{event_type}=\text{purchase}} \text{price}$$

$$\text{view_to_purchase_rate} = \frac{\text{purchase_count}}{\text{view_count}}$$

该模块的特点是直接、稳定、可解释，适合作为答辩时验证 Spark 读取与前端图表是否正常的的第一组证据。

4.5.4 转化分析模块

转化分析不直接按事件行计算，而是先把事件压缩成会话事实表。每个 `user_session` 只保留首次浏览、首次加购、首次购买时间，以及是否发生浏览、加购、购买。这样可以排除“购买时间早于浏览时间”这类顺序异常，避免把错误路径计入漏斗。合法路径定义如下：

$$\text{valid_cart_path} = 1 \iff t_{\text{view}} \neq \emptyset \wedge t_{\text{cart}} \neq \emptyset \wedge t_{\text{cart}} \geq t_{\text{view}}$$

$$\text{valid_cart_purchase_path} = 1 \iff t_{\text{view}} \leq t_{\text{cart}} \leq t_{\text{purchase}}$$

会话事实表与转化漏斗口径

```

1 session_facts = (
2     df.groupBy("user_session")
3     .agg(
4         F.min(F.when(F.col("event_type") == "view", F.col("event_timestamp"))).alias("
           ↳ first_view_ts"),
5         F.min(F.when(F.col("event_type") == "cart", F.col("event_timestamp"))).alias("
           ↳ first_cart_ts"),
6         F.min(F.when(F.col("event_type") == "purchase", F.col("event_timestamp"))).alias("
           ↳ first_purchase_ts"),
7         F.sum(F.when(F.col("event_type") == "purchase", F.coalesce(F.col("price"), F.lit
           ↳ (0))).otherwise(0)).alias("session_revenue"),
8     )
9     .withColumn("valid_cart_path", F.col("first_cart_ts") >= F.col("first_view_ts"))
10    .withColumn("valid_cart_purchase_path", F.col("first_purchase_ts") >= F.col("first_
           ↳ cart_ts"))
11 )
12
13 view_to_cart_rate = cart_sessions / view_sessions
14 cart_to_purchase_rate = purchase_sessions / cart_sessions

```

商品转化模块进一步把粒度降到 `product_id + user_session`，分别统计商品的浏览会话、加购会话、购买会话、销售额和转化率。这样前端可以展示“流量高但购买弱”的商品，也可以为后续运营优化模块提供候选基础。

4.5.5 行为路径模块

行为路径模块关注用户在一个会话中的行为顺序。实现上先对每个会话收集事件结构体，按事件时间排序，再把事件类型数组转为 `view → cart → purchase`

这样的路径签名。路径统计按签名聚合会话数、加购会话、购买会话、收入、平均步数和平均时长。转移统计则使用相邻事件数组：

行为路径与转移边生成

```

1 session_paths = (
2     cleaned_df.groupBy("user_session")
3     .agg(F.sort_array(F.collect_list(event_struct)).alias("event_structs"))
4     .withColumn("events", F.expr("transform(event_structs, x -> x.event_type)"))
5     .withColumn("path_signature", F.concat_ws(" -> ", F.expr("slice(events, 1, 8)")))
6 )
7
8 transitions = (
9     session_paths
10    .select(F.expr("arrays_zip(slice(events, 1, size(events)-1), slice(events, 2, size(
11        ↪ events)-1))").alias("pairs"))
12    .select(F.explode("pairs").alias("pair"))
13    .groupBy(F.col("pair.0").alias("from_event"), F.col("pair.1").alias("to_event"))
14    .count()
15 )

```

路径模块输出的 `exit_share` 是某个最后事件会话数除以总会话数，`conversion_rate` 是该路径或转移关联的购买会话数除以会话数。该口径用于定位“用户在哪一步退出”和“哪些路径更容易购买”。

4.5.6 特征集市模块

特征集市是后续异常、生命周期、实验模块的公共输入。它首先对清洗事件生成 `event_key`，其值为事件时间、事件类型、商品、用户、会话和价格拼接后的 SHA-256。随后按三种粒度聚合：

- 日商品特征：dt + product_id + brand + category_level1，统计 views、carts、purchases、unique users、unique sessions、revenue、avg price 与三种转化率。
- 日用户特征：dt + user_id，统计 sessions、views、carts、purchases、revenue、active minutes、触达商品数、触达类目数和偏好类目。
- 日类目特征：dt + category_level1，统计 views、carts、purchases、unique users、revenue、avg price 与 conversion rate。

质量指标中 $\text{duplicate_event_key_rate} = \text{duplicate_event_keys} / \text{event_rows}$ ， $\text{quarantined_rate} = \text{quarantined_rows} / \text{raw_rows}$ 。新鲜度指标用当前生成时间减去最大事件时间得到 `freshness_lag_hours`，并用最近 N 天窗口识别 late arrival。

4.5.7 预测模块

需求预测模块使用轻量的 rolling baseline，而不是在 driver 端训练复杂模型。Spark 先按全站和一级类目聚合每日需求：

$$\text{gmV}_{d,e} = \sum_{\text{date}=d, \text{entity}=e, \text{purchase}} \text{price}$$

系统只选择全站和 GMV 最高的 top 类目进入 driver，且通过 max_driver_history_rows 限制历史行数。预测时对最近训练窗口求 GMV 与购买数均值，再与上一周期均值比较，得到受限变化率：

$$\text{change_rate} = \frac{\overline{\text{gmV}}_{\text{recent}} - \overline{\text{gmV}}_{\text{previous}}}{\overline{\text{gmV}}_{\text{previous}}}$$

未来第 i 天的预测值为：

$$\widehat{\text{gmV}}_i = \max(0, \overline{\text{gmV}}_{\text{recent}} \times (1 + \text{clip}(\text{change_rate}, -0.25, 0.25) \times i/h))$$

若历史天数不足，则使用 sparse_baseline_fallback 并扩大预测区间。回测模块计算 WAPE，风险模块根据未来 GMV 相对近期均值的下降比例输出 high-/medium 风险。

4.5.8 关联图谱模块

关联图谱以会话内共同出现商品为基础。第一步把事件压缩为 product-session 表：同一会话同一商品只保留是否浏览、是否加购、是否购买、行为次数和收入。第二步选出收入、购买和浏览靠前的 top 商品，避免所有商品两两组合。第三步过滤商品数过多的 session，因为单个 session 有 n 个商品时会产生 $n(n-1)/2$ 条候选边。第四步自连接生成商品对，并分别计算 co-view、co-cart、co-purchase 三类关系。

边指标口径如下：

$$\text{support}(A, B) = |\{s : A \in s \wedge B \in s\}|$$

$$\text{confidence}(A \rightarrow B) = \frac{\text{support}(A, B)}{\text{sessions}(A)}$$

$$\text{lift}(A, B) = \frac{\text{support}(A, B) \times \text{total_sessions}}{\text{sessions}(A) \times \text{sessions}(B)}$$

affinity_quality 记录 eligible_session_count、pair_base_rows 和

pair_rows_per_input_row。如果这些指标异常升高，说明组合膨胀风险变大，需要收紧 top_nodes 或 max_pair_products_per_session。

4.5.9 Cohort 留存模块

Cohort 模块只使用购买事件。系统先找到每个用户的首次购买日期，并将首次购买月份作为 cohort；然后把用户每个月的购买聚合为 purchase cycle，计算当前购买月份与 cohort 月份之间的 period_index。留存矩阵中的核心指标为：

$$\text{retention_rate}_{c,p} = \frac{\text{active_users}_{c,p}}{\text{cohort_users}_c}$$

$$\text{repurchase_rate}_{c,p} = \frac{\text{repurchase_users}_{c,p}}{\text{cohort_users}_c}$$

价值曲线在留存矩阵上做窗口累计，得到 cumulative_revenue；复购间隔根据用户相邻购买周期差值分桶；cohort segment 按首次类目、复购率和收入识别高风险 cohort，并给出再营销建议。

4.5.10 商品组合与购物车挽回模块

商品组合模块先对事件补齐一级类目、品牌和价格带。价格带划分为 budget、mass、premium、luxury 和 unknown。类目、品牌、价格带分别聚合 views、carts、purchases、revenue、avg price 和转化率，并计算收入占比与购买占比。商品集中度使用收入份额平方作为 HHI 贡献：

$$\text{hhi_contribution}_i = \text{revenue_share}_i^2$$

购物车挽回模块把粒度设为 user_session + product_id。若商品在同一会话中出现加购事件，则进入购物车事实表；若后续存在购买且购买时间不早于首次加购，则 recovered=1；否则 abandoned=1。放弃金额计算为 cart_value，其中 cart_value = unit_price × cart_events。队列优先级为：

$$\text{priority_score} = \text{abandoned_value} \times \text{abandonment_rate}$$

如果移除率较高，推荐动作是检查商品摩擦；如果放弃率超过阈值，推荐动作是挽回提醒或优惠。

4.5.11 收入归因模块

归因模块先把每次购买生成唯一 `purchase_id`，再把同一会话中购买之前的 `view`、`cart` 和 `remove_from_cart` 事件作为触点。每个触点同时计算四种 `credit`：

- **First-touch**：第一个触点获得全部购买收入。
- **Last-touch**：最后一个触点获得全部购买收入。
- **Linear**：所有触点均分购买收入。
- **Time-decay**：越接近购买的触点权重越高，权重为 $\text{base}^{\text{reverse_position}-1}$ 。

收入归因 credit 计算

```
1 weighted = (
2     joined
3     .withColumn("touch_count", F.count("*").over(by_purchase))
4     .withColumn("position_before_purchase", F.row_number().over(asc))
5     .withColumn("reverse_position_before_purchase", F.row_number().over(desc))
6     .withColumn("linear_credit", F.col("purchase_revenue") / F.col("touch_count"))
7     .withColumn("first_touch_credit", F.when(F.col("position_before_purchase") == 1, F.
8         ↳ col("purchase_revenue")).otherwise(0.0))
9     .withColumn("last_touch_credit", F.when(F.col("reverse_position_before_purchase") ==
10        ↳ 1, F.col("purchase_revenue")).otherwise(0.0))
11     .withColumn("time_decay_weight", F.pow(F.lit(decay_base), F.col("reverse_position_
12        ↳ before_purchase") - 1))
13     .withColumn("time_decay_credit", F.col("purchase_revenue") * F.col("time_decay_weight
14        ↳ ") / F.sum("time_decay_weight").over(by_purchase))
15 )
```

之后系统分别按商品、品牌和类目聚合 `credit`，得到实体贡献；再统计高频路径和助攻机会。归因质量指标关注购买是否能找到触点，即 `attribution_coverage_rate`。

4.5.12 运营优化与推荐模块

运营优化模块的候选来自商品级转化统计。系统先计算 `views`、`carts`、`purchases`、`funnel purchases`、`revenue`、`avg price`、`revenue per view` 等指标。为了避免小样本商品因为少量购买被高估，购买率使用两层稳健化：一层是带先验权重的收缩购买率，另一层是 Wilson lower bound。候选置信度为：

$$\text{confidence} = \min(1, \sqrt{\text{views}/500}) \times (0.5 + 0.5 \times \min(1, \text{wilson/global_purchase_rate}))$$

优化目标是在总预算、运营位数量、类目上限和品牌上限约束下，选择“商品-动作”组合，使风险调整后的增量 `GMV` 最大。若本地没有 `Gurobi` 或求解失败，系统降级为贪心排序，仍输出可解释计划和 `degraded_greedy` 状态。

推荐模块复用商品转化特征与优化计划。商品分数由购买转化率、加购率和每浏览收入组成：

$$\begin{aligned} \text{product_score} = & 0.55 \times \text{view_to_purchase} + 0.25 \times \text{view_to_cart} \\ & + 0.20 \times \min(\text{revenue_per_view}/500, 1) \end{aligned}$$

目标会话按最近事件和偏好类目抽样。推荐时优先从用户偏好类目中选择未看过的高分商品；如果数量不足，再从优化计划或全局高分商品中兜底。最终按 **score** 和 **confidence** 排序，每个会话保留 **top-k**。质量报告统计覆盖会话数、**fallback** 率、平均置信度、重复推荐率和无效商品率。

推荐商品分与会话 top-k 计算

```
1 product_features = (
2     cleaned_df.groupBy("product_id", "brand", "category_level1")
3     .agg(
4         views.alias("views"),
5         carts.alias("carts"),
6         purchases.alias("purchases"),
7         revenue.alias("revenue"),
8     )
9     .withColumn("view_to_cart_rate", F.col("carts") / F.col("views"))
10    .withColumn("view_to_purchase_rate", F.col("purchases") / F.col("views"))
11    .withColumn("revenue_per_view", F.col("revenue") / F.col("views"))
12    .withColumn("product_score", F.col("view_to_purchase_rate") * 0.55
13        + F.col("view_to_cart_rate") * 0.25
14        + F.least(F.col("revenue_per_view") / 500.0, F.lit(1.0)) * 0.20)
15 )
16
17 rank_window = Window.partitionBy("user_session").orderBy(F.desc("score"), F.desc("
    ↳ confidence"))
18 recommendations = candidates.withColumn("rank", F.row_number().over(rank_window)).filter(
    ↳ F.col("rank") <= top_k)
```

4.5.13 异常检测与生命周期模块

异常检测模块复用特征集市中的日类目和日商品特征。每条监控信号由 **dt** + **entity_type** + **entity_id** + **metric** 组成，指标包括 **views**、**purchases**、**revenue**、**conversion rate** 等。系统使用中位数和 **MAD**，而不是均值和标准差，原因是电商数据常有促销尖峰和长尾商品，**MAD** 对极端值更稳健。**robust z-score** 计算为：

$$z_{\text{robust}} = \frac{|x - \text{median}(x)|}{1.4826 \times \text{MAD}(x)}$$

若历史点数不足，状态为 **watch**；若 **z-score** 超过 **warning** 或 **critical** 阈值，则生成对应告警；若当前值为 0 且基线量级高于最小流量阈值，也视为 **critical**。

生命周期模块以日用户特征为输入，按用户聚合活跃天数、会话数、浏

览数、加购数、购买数、收入、购买天数、最近活跃日期和偏好类目。分层规则是确定性的 RFM + engagement：高收入且多购买天数为 **champion**；达到高价值阈值为 **high_value**；有多日购买为 **loyal**；有购买为 **buyer**；只有加购为 **cart_intent**；只有浏览为 **browser**。风险带则由最近活跃天数和购买证据决定：超过 **at_risk_recency_days** 为 **at_risk**，无购买但有加购为 **convert_intent**。

4.5.14 实验分流与 Ops 证据模块

实验分流模块不直接在线投放，而是生成课程设计所需的可解释实验准备数据。实验目录包含生命周期再激活、推荐个性化和商品运营位三类实验。用户是否 **eligible** 由生命周期分层决定；进入实验后通过 `hash(user_id, experiment_key)` 生成稳定桶，桶值小于 **treatment split** 的用户进入 **treatment**，否则进入 **control**。这样同一份数据重复运行时，用户分组保持稳定。

实验 **guardrail** 检查包括 **treatment/control** 用户数、分组不平衡率、推荐覆盖率和优化计划是否存在。预期增量 **GMV** 不是实际因果效果，而是用于课程系统演示的计划值：

$$\text{expected_incremental_gmV} = \text{expected_uplift_rate} \times \max(\text{avg_order_value}, 50)$$

Ops 证据模块读取 **run manifest**、**SQLite job 表**、**Spark History 指标**和 **benchmark 结果**。它不重新计算业务指标，而是把一次运行的输入、输出、质量、耗时、**application id** 和存储清理策略串起来，形成答辩时可追溯的“证据链”。因此，业务页面回答“数据是什么”，Ops 页面回答“数据从哪里来、怎么算完、运行是否成功”。

4.6 前端 Ops 页实现

前端新增 `/api/v1/ops/evidence` 接口调用。后端 **BenchmarkEvidenceService** 读取正式 **benchmark** 目录，只返回精简字段：样本、对照组、耗时、吞吐、**YARN app id**、质量状态、**driver 峰值内存**和结果路径。前端 **Ops** 页新增两块内容：第一块是 **benchmark** 表，用于展示 1% 五组和 5% 两组实验；第二块是 **HDFS 输入与数据清理策略**，用于证明正式输入、**Parquet 层**和保留的 **Spark History app id**。

该设计解决了两个答辩痛点。第一，用户不需要在终端手动翻 **data/benchmarks**；第二，前端不会读取 **Spark stderr** 长文本，因此不会把 **History Server JSON** 限制问题转移到浏览器。核心接口返回结构如下：

```
{
  "benchmark_runs": [
    {
```

```

"sample": "1pct",
"variant": "yarn_only_csv",
"elapsed_seconds": 964.889,
"rows_per_second": 1143.869,
"spark_application_id": "application_1780991452919_0016",
"quality_status": "passed"
}
],
"hdfs_inputs": [...],
"cleanup_policy": {...}
}

```

图4.3展示了前端 Ops 页的证据集成方式。后端将 benchmark 结果、HDFS 输入快照、Spark History app id 和清理策略统一封装成轻量 JSON，前端只负责读取和展示这些证据，不直接解析 Spark 长日志。



图 4.3 前端 Ops 证据面板集成示意图（由 chat-gpt-image-2 生成）

4.7 旧数据清理实现

清理工作遵循“只删生成物，不删原始数据”的原则。保留的数据包括：

- data/raw/kaggle/ecommerce_behavior/2019-Oct.csv, 约 5.3GB。
- data/raw/kaggle/ecommerce_behavior/2019-Nov.csv, 约 8.4GB。
- data/sample/ecommerce_user_sample_1pct.csv, 约 141.5MB。
- data/sample/ecommerce_user_sample_5pct.csv, 约 694.3MB。
- 三个正式 benchmark 目录：1% fixed baseline、1% YARN matrix、5% YARN matrix。
- 最终 Spark History event log: application_1780991452919_0016 到 0021。

删除的数据包括早期 smoke、retry、history recheck benchmark 目录，旧 /user/course/ecommerce_behavior_processed 小输出，以及旧的大 Spark History 日志。清理后本地 benchmark 目录收敛到约 292KB，HDFS /spark-history 只保留 6 个最终 event log，约 87MB。存储快照如图4.4所示。

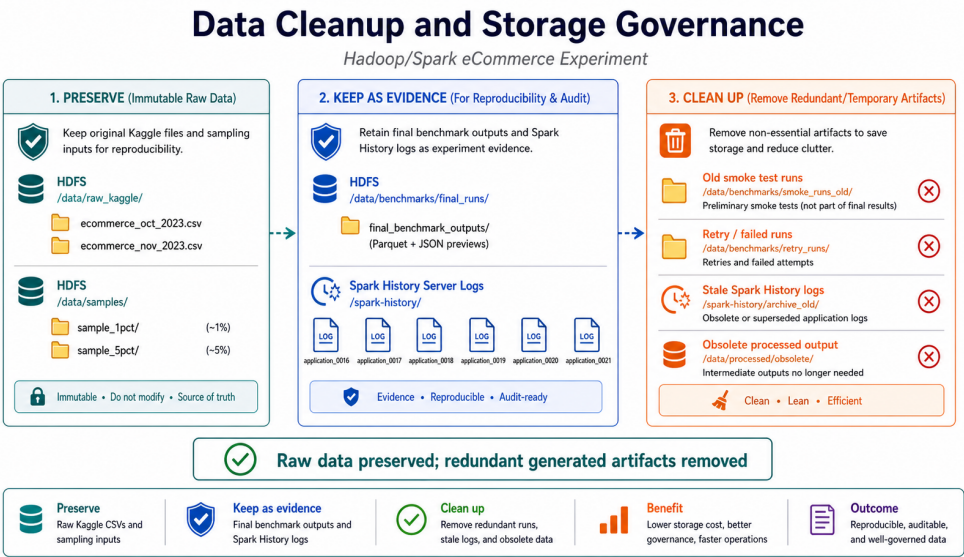


图 4.4 旧数据清理后的存储治理示意图（由 chat-gpt-image-2 生成，精确容量见正文说明）

4.8 实验结果

4.8.1 Benchmark 对照表

图4.5展示了本课题的实验对照矩阵。正式实验结果如表4.3所示。1% 样本包含五组对照，5% 样本包含算法优化 CSV 与 Parquet 两组压力验证。所有 YARN application 均为 SUCCEEDED。

表 4.3 1% 与 5% benchmark 实验结果

样本	对照组	耗时 (s)	输入行	输出行	Application ID
1%	Baseline local CSV	198.843	1,104,709	1,103,488	local-1781015058333
1%	YARN-only CSV	964.889	1,104,709	1,103,488	application_1780991452919_0016
1%	YARN + AQE CSV	538.657	1,104,709	1,103,488	application_1780991452919_0017
1%	YARN + algorithm CSV	535.804	1,104,709	1,103,488	application_1780991452919_0018
1%	YARN + Parquet	573.183	1,103,488	1,103,488	application_1780991452919_0019
5%	YARN + algorithm CSV	1875.099	5,412,207	5,405,748	application_1780991452919_0020
5%	YARN + Parquet	1828.329	5,405,748	5,405,748	application_1780991452919_0021

	1	2	3	4	5	结论
	Local CSV Baseline	YARN-only CSV	YARN + AQE	YARN + Algorithm Guards	YARN + Parquet	
输入数据	CSV 文件 (本地文件系统)	CSV 文件 (分布式存储)	CSV 文件 (分布式存储)	CSV 文件 (分布式存储)	Parquet 文件 (分布式存储)	YARN 只提供 资源调度；
调度方式	本地模式 (单机进程内)	YARN 调度 (资源分配与管理)	YARN 调度 (资源分配与管理)	YARN 调度 (资源分配与管理)	YARN 调度 (资源分配与管理)	AQE 与算法限流 降低 OOM 风险；
优化开关	AQE 关闭 算法限流 关闭	AQE 关闭 算法限流 关闭	AQE 开启 算法限流 关闭	AQE 开启 算法限流 开启	AQE 开启 算法限流 开启	Parquet 改善 列式读取与 落盘布局。
输出证据	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	

图 4.5 YARN + Spark 实验对照矩阵（由 chat-gpt-image-2 生成，精确数值见表4.3）

4.8.2 Spark History 指标

旧 Spark History Server 的 /stages API 在部分应用上会返回 500。为避免指标缺失，系统新增 scripts/collect_eventlog_metrics.py，直接解析 HDFS 中的 rolling event log .zstd 文件，汇总 task、shuffle 和 spill 指标。表4.4给出 6 个正式 YARN 应用的补采集结果。

表 4.4 Spark event log 指标补采集结果

Application ID	Tasks	Failed	Retried	Shuffle Read	Memory Spill
application_1780991452919_0016	6,290	0	0	9.4GB	2.1GB
application_1780991452919_0017	4,526	0	0	9.4GB	728.0MB
application_1780991452919_0018	4,526	0	0	9.4GB	728.0MB
application_1780991452919_0019	5,057	0	0	9.4GB	792.5MB
application_1780991452919_0020	5,149	0	0	44.9GB	89.6GB
application_1780991452919_0021	5,815	0	0	44.9GB	89.4GB

从表中可以看到，正式 YARN 作业均没有 failed task 和 retried task。5% 样本的 spill 显著高于 1% 样本，说明本机 Docker 资源已经进入更明显的磁盘/内存交换区间；这也支持本文“不盲目承诺全量无条件通过，而用典型部分数据证明优化有效”的结论。

4.8.3 典型模块 Benchmark

根据课程实验目标与本机资源约束，后续不再启动全量 Oct+Nov 长任务，而是使用 1% 样本中的 200,000 行典型数据补跑模块级 benchmark。该数据规模足以

覆盖关联、推荐、异常和实验分流的核心计算路径，同时避免全量运行对 Docker Desktop 内存与磁盘造成不必要压力。结果如表4.5所示。

表 4.5 典型部分数据模块 benchmark 结果

模块	任务	输入行	输出行	耗时 (s)
Affinity	affinity_pipeline	199,895	80	3.852
Recommendation	recommendation_pipeline	199,895	1,000	1.740
Anomaly	anomaly_pipeline	199,895	644	7.865
Experimentation	experimentation_pipeline	199,895	21,716	8.703

4.8.4 结果分析

图4.6以生成式信息图方式概括了 benchmark 证据，精确数值以表4.3为准。1% 样本中，YARN-only CSV 用时 964.889s，明显慢于 local baseline 和后续优化组。这说明 YARN 本身提供资源调度和容错框架，但不能自动减少 CSV 扫描、shuffle、driver collect 和算法组合膨胀。启用 AQE 后耗时降至 538.657s，相对 YARN-only 约有 1.79 倍改善；加入算法限流后耗时为 535.804s，相对 YARN-only 约有 1.80 倍改善。

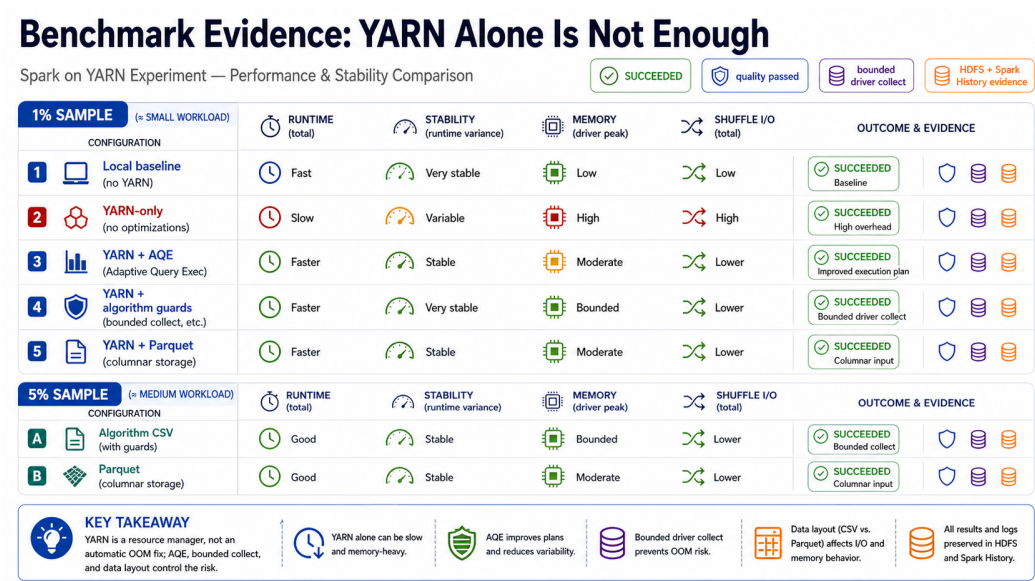


图 4.6 Benchmark 证据概览（由 chat-gpt-image-2 生成，精确数值见表4.3）

5% 样本中，YARN + algorithm CSV 用时 1875.099s，YARN + Parquet 用时 1828.329s。两组都成功完成，说明 driver collect 收敛和算法上界对稳定性有效。Parquet 组在 5% 样本中略快，说明列式输入在更大样本上开始体现优势；但由于本机 Docker 资源有限、YARN 调度开销和重复 pipeline 模块仍然存在，Parquet 收益并非线性放大。

4.8.5 实验结论

从实验结果可以得到三点结论。第一，仅将 **Spark** 提交到 **YARN** 不会自动解决性能和内存问题，**YARN-only** 组反而因为调度、容器和 **HDFS** 访问开销表现最慢。第二，**AQE** 和算法层限流是关键优化，能显著降低不必要 **shuffle**、单分区窗口和 **driver** 端大收集风险。第三，**Parquet** 输入是后续扩展到更大数据规模的必要基础，但需要与分区、列裁剪和模块级缓存一起使用。

因此，本课题最终结论应表述为：在本机 **ARM64 Docker Hadoop YARN** 环境中，系统已经显著降低 **OOM** 风险，并通过 **1%/5% pipeline benchmark**、**event log** 指标和 **20 万行典型模块 benchmark** 给出证据。按本次实验取舍，不再强行运行全量 **Oct+Nov**；全量处理可作为后续资源边界扩展，在更高 **Docker** 内存、更多 **executor** 或云端集群环境中继续验证。

第 5 章 致谢

本课程设计能够完成，首先感谢任课老师在大数据与云计算课程中对 Hadoop、Spark、分布式存储和集群实验方法的讲解，使本项目能够从普通本地数据分析扩展到 Hadoop YARN 实验环境。老师在课程中强调实验证据、运行截图和结果解释的重要性，也促使本项目不只停留在功能演示，而是进一步补充 benchmark、质量门禁和 Ops 展示。

感谢同学们在调试 Docker、HDFS、YARN、Spark submit、前端页面和实验报告过程中提供的交流与帮助。尤其是在 ARM Mac 架构、Docker 镜像选择、端口冲突、Spark History 日志异常和前端展示问题上，同学之间的讨论帮助我更快定位了问题边界。

同时感谢开源社区提供的 Hadoop、Spark、Docker、Flask、React、Vite、TanStack Query、Matplotlib 等工具。没有这些成熟工具，本课程设计很难在本机环境中完成从原始数据、分布式计算、后端 API 到前端运维展示的完整闭环。

参考文献

- [1] Dean J, Ghemawat S. Mapreduce: Simplified data processing on large clusters[C]//Proceedings of the 6th Symposium on Operating Systems Design and Implementation. 2004: 137-150.
- [2] Vavilapalli V K, Murthy A C, Douglas C, et al. Apache hadoop yarn: Yet another resource negotiator[C]//Proceedings of the 4th Annual Symposium on Cloud Computing. 2013: 1-16.
- [3] Zaharia M, Chowdhury M, Das T, et al. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing[C]//Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation. 2012: 15-28.
- [4] Zaharia M, Xin R S, Wendell P, et al. Apache spark: A unified engine for big data processing [J]. Communications of the ACM, 2016, 59(11): 56-65.
- [5] Armbrust M, Xin R S, Lian C, et al. Spark sql: Relational data processing in spark[C]//Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data. 2015: 1383-1394.
- [6] Herodotou H, Lim H, Luo G, et al. Starfish: A self-tuning system for big data analytics[C]//Proceedings of the 5th Biennial Conference on Innovative Data Systems Research. 2011: 261-272.
- [7] Kwon Y, Balazinska M, Howe B, et al. Skewtune: Mitigating skew in mapreduce applications[C]//Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data. 2012: 25-36.
- [8] Rasmussen A, Porter G, Conley M, et al. Themis: An i/o-efficient mapreduce[C]//Proceedings of the Third ACM Symposium on Cloud Computing. 2012: 13:1-13:14.
- [9] Docker Inc. Docker compose documentation[EB/OL]. 2026. <https://docs.docker.com/compose/>.
- [10] Apache Hadoop Project. Hdfs architecture[EB/OL]. 2026. <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>.
- [11] Apache Hadoop Project. Apache hadoop yarn[EB/OL]. 2026. <https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/YARN.html>.
- [12] Apache Spark Project. Running spark on yarn[EB/OL]. 2026. <https://spark.apache.org/docs/3.5.5/running-on-yarn.html>.
- [13] Apache Spark Project. Spark sql performance tuning[EB/OL]. 2026. <https://spark.apache.org/docs/3.5.5/sql-performance-tuning.html>.
- [14] Apache Parquet Project. Apache parquet documentation[EB/OL]. 2026. <https://parquet.apache.org/docs/>.
- [15] Apache Spark Project. Spark configuration[EB/OL]. 2026. <https://spark.apache.org/docs/3.5.5/configuration.html>.